

1 Costruzione

1. Definire confini del sistema;
2. Identificare attori;
3. Identificare casi d'uso;
4. Definire diagramma;
5. Strutturare casi d'uso;
6. Descrivere casi d'uso in linguaggio naturale o con diagrammi comportamentali.

1.1 Definire confini

Quali responsabilità rientrano nei confini del sistema che stiamo modellando?

1.2 Identificare attori

Identificare attori che interagiscono con il sistema per eseguire qualche compito (attori che necessitano del sistema per svolgere qualche compito e attori cui il sistema si rivolge per svolgere qualche compito), gli attori vanno poi raggruppati secondo ruoli (responsabilità) rispetto al sistema. Vengono anche identificati altri sistemi SW e dispositivi esterni che interagiscono con il sistema per svolgere qualche compito.

1.3 Identificare casi d'uso

Per ogni attore:

- Si identificano compiti e funzioni (di basso livello che l'attore deve essere in grado di eseguire tramite il sistema e compiti che sistema richiede che l'attore esegua);
- Si raggruppano compiti e funzioni in casi d'uso (devono corrispondere ad un obiettivo specifico per attore o sistema, raggruppano funzioni eseguite in sequenza o innescate dallo stesso evento, deve essere né troppo grande né troppo piccolo);
- Si dà un nome al caso d'uso sintetizzando la funzionalità svolta;
- Si definisce il diagramma dei casi d'uso:
 - Deve contenere relazioni fra attori e casi d'uso;
 - Ogni attore deve partecipare a 1+ casi d'uso;
 - Ogni caso d'uso deve avere almeno un attore con cui comunica;
 - Se due attori partecipano a stessi casi d'uso si considera possibilità di combinarli in un unico attore
- Si strutturano casi d'uso:
 - Si identificano relazioni di estensione (specializzando scenari alternativi e collegando nuovi casi d'uso a quelli di partenza mediante relazione `extend()`);
 - Si identificano relazioni di inclusione (estraendo parti comuni in casi d'uso diversi e collegando casi d'uso che condividono una parte in comune al nuovo caso d'uso rappresentante il comportamento condiviso mediante relazione `include()`).

2 Concetti fondamentali

2.1 Diagramma UML delle classi

- Diagramma **strutturale**;
- Mostra una vista **statica** del modello (mostra organizzazione del progetto del sistema ma *non mostra informazioni temporali*);
- Suoi elementi corrispondono a concetti fondamentali del paradigma OO;
- Il più diffuso attualmente;
- Rappresenta:
 - tipi di oggetti (classi) del sistema che corrispondono a entità esistenti nel sistema;

- relazioni *statiche* tra classi e vincoli che si applicano a tali relazioni;
- caratteristiche delle classi (proprietà = attributi e responsabilità = metodi);

2.2 Diagramma delle classi nel processo SW

Può essere utilizzato nella fase di definizione requisiti, consente una vista concettuale delle **entità** nel dominio del problema e cattura concetti principali del dominio fissando i confini del sistema. Semplice modello architetturale rivolto a comprensione di cosa fa il sistema, non di come lo fa.

Può essere usato in implementazione, identificando **classi** del SW e relazioni tra esse, corrisponde a reale struttura del SW poiché ne modella implementazione OO

3 Sintassi

3.1 Classe

Descrittore di un insieme di oggetti con stesse proprietà (attributi), comportamento (operazioni) e relazioni con altri oggetti, con lo stesso concetto della programmazione OO.

- Ogni oggetto è istanza di una sola classe;
- In un dato istante un oggetto ha uno specifico stato (valore dei suoi attributi);
- In base al proprio stato due oggetti possono rispondere diversamente alla stessa operazione.

Una classe è rappresentata da un rettangolo con tre sottosezioni: nome della classe e, opzionalmente (a seconda del livello di dettaglio desiderato), attributi e operazioni. Il nome è al **singolare** e solitamente in **UpperCamelCase**.

Inserisci immagine

3.2 Attributi

3.2.1 Visibilità

Sono ammessi tre livelli di **visibilità** degli attributi:

- **+** **pubblico**: accesso esteso a tutte le altre classi;
- **#** **protetto**: accesso consentito soltanto a classi che derivano dalla classe originale;
- **-** **privato**: soltanto la classe originale può accedere a tali attributi.

Il **nome** dell'attributo è l'unico parametro obbligatorio, mentre il **tipo** può essere un tipo primitivo (int, double...) oppure il nome di una classe definita nello stesso diagramma (in tal caso l'attributo può essere indicato con un'*associazione*), esso definisce un vincolo sugli oggetti che possono corrispondere all'attributo.

3.2.2 Molteplicità

La **molteplicità** indica il quantitativo degli attributi di un certo tipo, indicando attributi come array o matrici con valore di default 1, il numero minimo e massimo possono essere racchiusi tra parentesi quadre:

- **[1..1]** esattamente 1;
- **[0..1]** al più 1;
- **[0..]** numero imprecisato;
- **[1..]** almeno 1.

Gli elementi di una molteplicità a più valori sono considerati come un *insieme*, non ordinati (in caso contrario notazione **{ordered}**) e non duplicati (in caso contrario notazione **{nonunique}**).

3.2.3 Valori di default e proprietà

- **valueDefault** è valore assegnato all'attributo di default, in caso non ne vengano specificati altri durante la creazione;
- **{proprietà}** indica caratteristiche aggiuntive dell'attributo (ad esempio sola lettura).

3.3 Operazioni

Corrispondono a metodi di una classe (operazioni invocabili sugli oggetti istanza della classe), sono azioni che possono essere svolte dalla classe stessa e, solitamente, nei diagrammi non vengono riportate le operazioni che si occupano solamente di modificare attributi (in quanto facilmente deducibili).

- Operazioni invocabili sugli oggetti istanza della classe;
- **Visibilità** e **nome** analoghi agli attributi;
- **tipoRestituito** valore di ritorno;
- **listaParametri** contiene nome e tipo dei parametri nella forma *direzione nome parametro: tipo = valoreDefault*, con:
 - **direzione**: input (in), output (out) o entrambi (inout), valore di default è in;
 - **nome**, **tipo** e **valoreDefault** analoghi a quelli degli attributi.

3.4 Responsabilità

Solitamente nei diagrammi *concettuali* che descrivono dominio del sistema non si inseriscono metodi in quanto rappresenterebbero un elemento del dominio della soluzione, in alternativa si possono usare le **responsabilità** (insieme di funzioni principali che classe dovrebbe garantire, utili per controllare completezza del modello di dominio)

3.5 Note / Testo descrittivo

Commenti aggiuntivi in linguaggio naturale che possono essere utilizzati in in ogni tipo di diagramma UML, simili a commenti in un linguaggio di programmazione. *Inserisci immagine*

4 Associazioni

Esprime una relazione (fisica o concettuale) tra istanze (oggetti) delle classi, caratterizzata da:

- **nome**: esprime legame semantico tra classi associate;
- eventuale **ruolo** giocato da ciascuna delle parti associate;
- **molteplicità** dell'associazione;
- **verso di navigazione** dell'associazione.

4.1 Nome

Etichetta dell'associazione, solitamente un verbo, permette di formare frasi di senso compiuto (es. "Una persona possiede un'automobile"), per evitare ambiguità si può specificare direzione dell'associazione.

4.2 Ruolo

Si può specificare **ruolo** che oggetti di una classe rivestono quando sono collegati da istanze dell'associazione, il ruolo viene specificato sull'estremità dell'associazione prossima alla classe.

4.3 Molteplicità

Vincola numero di oggetti di una classe che possono partecipare a una associazione in ogni istante, sono riportate sull'estremità dell'associazione prossima alla classe nella forma **minimo..massimo**

4.4 Verso di navigazione

Indica in che direzione è possibile reperire le informazioni: utile soprattutto nel **progetto di dettaglio**, rispecchiando una scelta di progetto, non presente (di solito) nei **progetti concettuali**.

4.5 Riepilogo

Inserisci Immagine

4.6 Associazioni vs Attributi

- Caratteristiche strutturali di una classe devono essere rappresentate sia come associazioni che come attributi di una classe;
- A differenza dagli attributi, associazioni possono riportare anche molteplicità di entrambe le classi ma sono meno compatte;
- Progettista deve scegliere entità che hanno maggiore importanza e rappresentarle come classi per dar loro più enfasi;
- Consigliabile non appesantire il diagramma con troppe associazioni 1 a 1;
- Si possono usare **attributi** per concetti secondari (come tipi semplici) e **associazioni** per classi più significative (la scelta dipende da cosa si vuole enfatizzare nel diagramma).

4.7 Implementazione

La rappresentazione più comune di un'associazione o un'attributo di un'entità in linguaggi di programmazione OO è tramite un attributo di una classe, se definito un verso di navigazione, la classe origine ha un attributo con:

- Nome: ruolo della destinazione;
- Tipo: la classe di destinazione;
- Molteplicità: la molteplicità della destinazione.

4.7.1 Uno a Molti

Esempio: una Persona, molte Automobile

- La classe Persona ha un attributo vettore di Automobile;
- La classe Automobile ha un attributo Persona;
- Entrambe: tutte e due le classi permettono di navigare l'altra classe dell'associazione (nessuna freccia o doppia freccia).

4.7.2 Uno a Uno

Esempio: uno Studente, un Badge

- La classe Badge è implementata avendo un attributo Studente;
- La classe Studente è implementata avendo un attributo Badge;
- Entrambe se bidirezionale.

4.7.3 Molti a Molti

Esempio: molti Studente, molti Esame

- La classe Studente implementata avendo un attributo array di Esame;
- La classe Esame implementata avendo anche un attributo array di Studente;
- Entrambe se bidirezionale.

5 Associazioni Riflessive e Classi Associative

5.1 Associazioni Riflessive

Associazione che collega una classe con sé stessa (oggetti di una classe possono quindi essere associati ad oggetti appartenenti alla classe stessa). Una cartella può contenere 0 o più sottocartelle (ipoteticamente infinite), una cartella può essere contenuta al massimo in una sola altra cartella.

5.2 Classi Associative

Classe che connette due altre classi e definisce un insieme di caratteristiche proprie della associazione stessa, ognuna delle sue istanze rappresenta un'associazione tra due istanze delle classi e contiene specifici valori degli attributi. Dato che caratterizza un'associazione prima di creare un'istanza della classe associativa vanno create le istanze delle classi collegate.

- Rappresentano collegamenti concettuali;
- Ci può essere solo un'istanza di una specifica classe associativa tra ogni coppia di soggetti associati;
- Per *reificare* (rendere reale) una classe associativa sarà opportuno trasformarla in una classe.

inserisci immagine

6 Generalizzazione - Specializzazione

I concetti di generalizzazione e specializzazione in UML sono analoghi a quelli di OO, indicano legami del tipo *is-a* tra le sottoclassi e la superclasse (quest'ultima è un concetto generale, le sottoclassi sono due specializzazioni).

- Tipo di associazione rappresentato da una freccia che va da classe specializzata a classe generale;
- Informazioni comuni solitamente mantenute in un solo posto (classe generale), facilitando eventuali modifiche;
- Sottoclassi hanno tutti gli attributi e le operazioni delle classi generali, aggiungendone di più specifici;
- A livello **concettuale** (analisi requisiti), gen-spec esprime una relazione *is-a* tra un concetto generale e sue specializzazioni;
- A livello di **dettaglio** (progetto / implementazione) può essere interpretato come:
 - relazione di **ereditarietà** tra due classi concrete (classi derivate ereditano attributi e metodi public / protected della classe base);
 - relazione di **realizzazione** tra una classe astratta e una classe concreta (classe base ha solo prototipi di metodi, quella specializzata implementa metodi sfruttando overriding).

6.1 Esempio

Inserisci Immagine

7 Contenimento

Tipo speciale di associazione tra classi che rappresenta il legame tra un insieme (il "tutto") e le sue parti, ovvero tra un contenitore e il contenuto, questo può essere:

- debole, detto **aggregazione**, se la parte mantiene la sua identità quando entra a far parte del tutto (auto - motore);
- stretto, detto **composizione**, se la parte perde la sua identità entrando a far parte del tutto (pane - farina).

7.1 Aggregazione

- Lato del "tutto" è detto **aggregato**;
- Parti possono esistere indipendentemente dall'aggregato;
- Possibile che più aggregati condividano una stessa parte;
- Molteplicità dal lato dell'aggregato, quando sottintesa, vale 0..1;
- **Ciclo di vita** dell'oggetto contenuto è indipendente da quello del contenitore;
- Oggetto contenitore non necessariamente responsabile della costruzione e distruzione dell'oggetto contenuto.

7.2 Composizione

- Parti non esistono senza il tutto: se composito viene distrutto anche le sue parti verranno distrutte (o responsabilità viene ceduta ad un altro oggetto);
- Composito è responsabile di costruzione / distruzione degli oggetti contenuti;
- Ogni parte può appartenere ad un solo composito per volta;
- Una **classe** può essere componente di più classi, ma un'**istanza** di classe componente può essere componente di un solo oggetto composito.
- Molteplicità dal lato del "tutto", quando sottintesa, vale 1 nel caso in cui classe componente ha solo un'altra classe composito (altrimenti se classe componente può essere contenuta in due o più classi composito diverse la molteplicità è 0..1).